

FoveaMap — Product Requirements Document

Variable-resolution 2.5D semantic mapping from Lidar · Sep 30, 2026 · @Ammar Hashmi

1. Overview

FoveaMap turns each raw Lidar sweep into a compact, semantically labelled 2.5D elevation map whose cells are 5 cm near the vehicle and 50 cm at 100 m, cutting map memory about 50x versus a uniform 5 cm grid while keeping safety-critical detail where it matters.

The problem. Autonomous vehicles need to know, every 100 ms, where they can drive and what is around them.

- **Dense 3D point clouds** carry the detail (curbs, potholes, overhangs) but a 128-beam sensor produces hundreds of thousands of points per sweep. Processing and storing them uniformly creates compute and memory-latency bottlenecks.
- **2D occupancy grids** are cheap but throw away height, so a 12 cm curb, a pothole or a low-hanging branch looks the same as flat road.
- **Uniform high-resolution grids** waste most of their cells: at 50 m and beyond a Lidar returns only a few points per square metre, so fine cells stay empty or noisy.

The idea. Mimic human foveated vision. Spend resolution where points are dense and decisions are urgent (the first 10 m), and coarsen it smoothly with distance, where the sensor itself delivers less detail.

What we ship. A software framework with four parts: a point-cloud semantic and motion segmentation network, a variable-resolution 2.5D grid engine, a real-time visualization dashboard, and a benchmark harness that reports latency, memory and accuracy by distance band.

2. Goals and non-goals

Goals

1. **G1 Terrain analysis.** Label every observed cell as drivable, non-drivable ground, or unknown, with per-cell height and roughness.
2. **G2 Object understanding.** Segment points into static obstacles (walls, poles, vegetation, buildings) and dynamic agents (pedestrians, cyclists, vehicles), and flag which are moving.

3. **G3 Foveated representation.** Project classified points into a variable-resolution 2.5D grid (5 cm near, 50 cm far, 100 m radius) with no misalignment or dropped points at tier boundaries.
4. **G4 Real time.** Run the full pipeline at the sensor rate (10 Hz) or faster, with a target of 20 FPS on a single desktop GPU.
5. **G5 Proof.** Show, with measured numbers, the memory saving over uniform grids and the accuracy at each distance band.

Non-goals (v1)

- Path planning, control or actuation. FoveaMap produces the map; a planner consumes it.
- Camera or radar fusion. The interface leaves room for it (see Architecture Vision), but v1 is Lidar-only.
- Global SLAM or loop closure. v1 uses the ego-pose supplied by the dataset or an odometry source.
- Object tracking with persistent IDs and trajectory prediction.
- Certification to automotive safety standards (ISO 26262). v1 is a research and demo framework.

3. Users and operating contexts

Persona	What they need from FoveaMap	Primary surface
Planning engineer	A stable map API: traversability cost, height and class per cell, refreshed every frame	ROS 2 topic / Python API
Perception researcher	Per-class and per-distance accuracy, easy model swapping, reproducible benchmarks	Benchmark harness, config files
Systems / embedded engineer	Predictable latency and memory, per-stage timings, GPU utilisation	Metrics panel, profiler logs
Evaluator or judge	A clear live demo proving detail near, savings far, and real-time speed	Dashboard

Operating contexts in scope: urban streets and campus roads, day and night (Lidar is light-independent), speeds up to 50 km/h, flat and moderately sloped terrain. Heavy rain, snow and dust are recorded as known degradations, not v1 targets.

4. User stories

ID	As a...	I want...	So that...
US-1	Planning engineer	curbs, potholes and kerb drops within 10 m resolved at 5 cm with their height step	the planner never routes a wheel over a 10 cm edge
US-2	Planning engineer	each cell to carry a traversability cost and an unknown flag	unobserved space is treated as risky, not free
US-3	Planning engineer	overhanging obstacles (branches, barriers, truck beds) kept as a clearance height	a low bridge is not mistaken for open road
US-4	Planning engineer	moving pedestrians and vehicles on a separate dynamic layer	static layers do not smear moving objects into ghost walls
US-5	Perception researcher	to swap the backbone (SPVCNN, MinkUNet, PointNet++) through config	I can compare models without touching the grid engine
US-6	Perception researcher	accuracy reported per class and per distance band	I can see where the model degrades with range
US-7	Systems engineer	per-stage latency and memory in every frame's metrics	I can find and fix the bottleneck
US-8	Evaluator	to toggle between foveated and uniform grids live, with memory shown side by side	the saving is visible, not just claimed
US-9	Evaluator	to replay a recorded drive and scrub through frames	I can inspect a failure case at my own pace

5. Functional requirements

P0 = required for the v1 demo, P1 = should have, P2 = stretch.

ID	Module	Requirement	Priority
FR-1	Ingestion	Read point clouds from KITTI .bin, nuScenes, PCD files and a live ROS 2 <code>PointCloud2</code> topic with x, y, z, intensity and timestamp	P0
FR-2	Ingestion	Transform each sweep into the vehicle frame with the supplied ego-pose; de-skew motion distortion when per-point timestamps exist	P1
FR-3	Segmentation	Classify every point into at least 8 classes: road, sidewalk, other-ground, vegetation/terrain, building/wall, pole/sign, vehicle, person/cyclist	P0
FR-4	Segmentation	Output a per-point moving / static flag using the current and previous sweeps	P0
FR-5	Segmentation	Output per-point confidence (softmax probability) for use in grid fusion	P1
FR-6	Segmentation	Backbone selectable by config: sparse 3D conv (default), point-based (PointNet++), range-image (fallback for low-power)	P1
FR-7	Grid engine	Build a concentric, multi-tier grid: 5 cm cells within 10 m, 50 cm cells to 100 m, tier sizes and radii configurable	P0
FR-8	Grid engine	Every coarse cell is an exact union of fine cells (integer size ratio, shared origin), so no point is lost or double-counted at a tier edge	P0
FR-9	Grid engine	Store per cell: min, max and mean height, height variance, point count, class histogram, dominant class, dynamic flag, clearance height, last-seen time	P0
FR-10	Grid engine	Derive a traversability cost per cell from class, slope, step height and roughness	P0
FR-11	Grid engine	Keep the grid centred on the vehicle by scrolling (ring buffer), not by rebuilding	P1
FR-12	Grid engine	Fuse static layers over time with confidence weighting; clear the dynamic layer each frame	P1
FR-13	Grid engine	Query API: height, class and cost at a world point; ray-cast; export as ROS <code>grid_map</code> or NumPy	P1

ID	Module	Requirement	Priority
FR-14	Dashboard	Show the live 2.5D map (top-down and 3D), colour-coded by class, with tier boundaries visible	P0
FR-15	Dashboard	Show FPS, per-stage latency, map memory, and a side-by-side memory comparison against uniform 2.5D and 3D grids	P0
FR-16	Dashboard	Toggle layers (semantic, elevation, traversability, dynamic, confidence) and switch foveated vs uniform view	P1
FR-17	Dashboard	Replay recorded sequences with play, pause, step and scrub	P1
FR-18	Benchmarking	A single command runs a sequence and writes latency, memory and accuracy (per class, per distance band) to JSON and a report	P0
FR-19	Benchmarking	Compare against baselines: uniform 5 cm 2.5D grid, uniform 5 cm voxel grid, and 2D occupancy grid	P0
FR-20	Deployment	Export the model to ONNX / TensorRT with FP16 inference	P2

6. Non-functional requirements

The pipeline must finish each frame inside one sensor period (100 ms at 10 Hz) with headroom; the target is 50 ms end to end.

ID	Area	Requirement	Target	Hard limit
NFR-1	Latency	Sweep in to map out, p95, on reference GPU	≤ 50 ms	≤ 100 ms
NFR-2	Throughput	Frames processed per second, sustained	≥ 20 FPS	≥ 10 FPS
NFR-3	Memory	Foveated map, 100 m radius, all layers	≤ 8 MB	≤ 16 MB
NFR-4	Memory	Saving versus uniform 5 cm 2.5D grid of the same extent	≥ 30x	≥ 20x
NFR-5	GPU memory	Peak VRAM for model + grid	≤ 4 GB	≤ 6 GB

ID	Area	Requirement	Target	Hard limit
NFR-6	Accuracy	Point segmentation mIoU, 0–10 m band	≥ 70%	≥ 60%
NFR-7	Accuracy	Drivable-surface IoU on the grid, 0–10 m	≥ 90%	≥ 85%
NFR-8	Fidelity	Height step detection (curb ≥ 8 cm) recall within 10 m	≥ 90%	≥ 80%
NFR-9	Robustness	Frame drop or sensor gap does not crash the pipeline; map marks stale cells	always	always
NFR-10	Determinism	Same input and seed give identical grid output	always	always
NFR-11	Portability	Runs on Linux x86 + NVIDIA GPU; stretch: Jetson Orin	x86 in v1	x86

Reference hardware: a single NVIDIA RTX 3060–4070 class desktop GPU with 12 GB VRAM, 8-core CPU, 32 GB RAM. Every reported number states the hardware it was measured on.

7. Success metrics and acceptance criteria

v1 is accepted when one command reproduces all five checks below on a held-out sequence and the dashboard shows them live.

- ☐ **Speed:** p95 end-to-end latency ≤ 50 ms and ≥ 20 FPS sustained over a 1,000-frame sequence.
- ☐ **Memory:** foveated map ≤ 8 MB and at least 30x smaller than a uniform 5 cm 2.5D grid of the same 100 m extent (measured bytes, not theoretical).
- ☐ **Accuracy near:** mIoU ≥ 70% and drivable IoU ≥ 90% in the 0–10 m band.
- ☐ **Accuracy with range:** mIoU reported for 0–10, 10–25, 25–50 and 50–100 m, with the drop from the first to the last band explained.
- ☐ **Alignment:** zero points lost at tier boundaries (points in = points binned, checked every frame) and no visible seam in height at 10 m.

Metric definitions

- **mIoU** — mean over classes of $TP / (TP + FP + FN)$, computed on points and, separately, on grid cells against a rasterised ground truth.
- **Distance-banded accuracy** — the same metrics restricted to points or cells whose centre falls in each radial band.

- **Moving-object IoU** — IoU of the moving class alone, as in SemanticKITTI's moving object segmentation task.
- **Latency** — wall-clock from sweep arrival to map publish, with GPU synchronised; p50, p95, p99.
- **Map memory** — bytes actually allocated for all grid layers, read from the allocator, plus a theoretical cells × bytes-per-cell figure.
- **Memory saving** — uniform-grid bytes ÷ foveated-grid bytes for the same radius and layers.

8. Data and evaluation protocol

Train and report on SemanticKITTI first, because its multi-scan labels separate moving from static objects, which covers G1, G2 and FR-4 with one dataset.

Dataset	Why we use it	Role
SemanticKITTI	64-beam Lidar, per-point labels, moving vs static classes, standard train/val split (sequence 08 = val)	Primary train + eval
nuScenes-lidarseg	32-beam sensor, different city and density; tests generalisation	Secondary eval, fine-tune
Own or recorded bag (optional)	Proves the ROS 2 path on real hardware	Live demo

Class mapping. Dataset classes are merged into the FoveaMap taxonomy: *drivable* (road, parking, lane marking), *non-drivable ground* (sidewalk, terrain, other-ground), *static obstacle* (building, fence, wall, pole, sign, trunk, vegetation), *vehicle*, *person / cyclist*, each vehicle and person class split into moving and static.

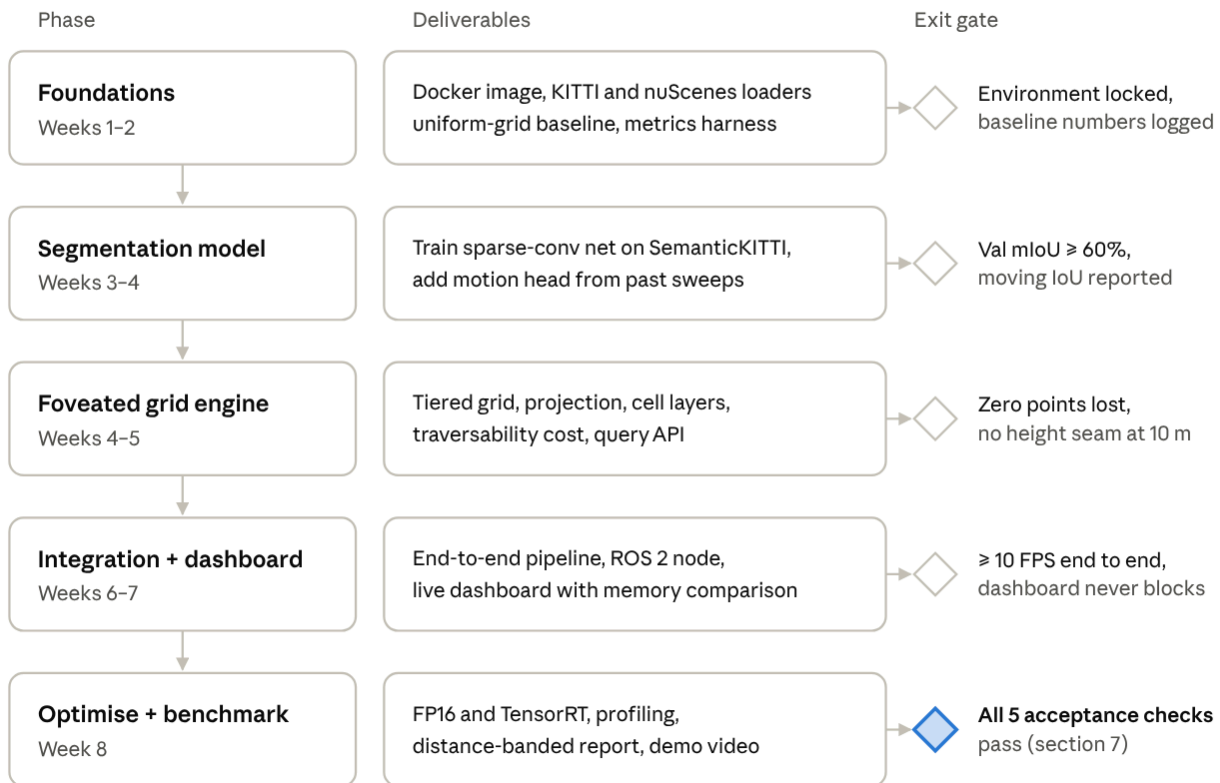
Ground truth for the grid. Rasterise labelled points into the same foveated grid; a cell's true class is the majority label and its true height comes from the labelled ground points. Cells with fewer than 3 points are scored as unknown.

Protocol. Fixed random seeds, three runs per model, report mean and spread. Every result table lists distance band, hardware and model checkpoint.

9. Scope and milestones

v1 is an eight-week build; each phase ends at a gate that must pass before the next phase's work is trusted.

Eight weeks from data loaders to a benchmarked real-time demo



v1 roadmap · 5 phases, 5 exit gates, 8 weeks

Grid-engine work starts in week 4 against ground-truth labels, so it does not wait for the model. Post-v1 candidates: camera fusion, Jetson Orin port, object tracking with IDs, learned traversability.

10. Risks, assumptions and open questions

Risk	Impact	Mitigation
Accuracy collapses beyond 50 m because points are sparse	Far objects misclassified	Report by band; use cell-level voting with confidence; coarse cells need fewer points to decide
Seams at the 10 m tier boundary (height jumps, double counting)	Planner sees phantom steps	Integer size ratios + shared world origin; boundary unit tests on every commit
Sparse conv libraries (spconv, TorchSparse, MinkowskiEngine) are hard to install	Lost days on setup	Pin CUDA and library versions in a Docker image from week 1; PointNet++ as fallback

Risk	Impact	Mitigation
Model inference exceeds 30 ms budget	FPS target missed	FP16, TensorRT, crop to 100 m, voxel size 5–10 cm at input, smaller channel widths
Moving vs static confusion for parked cars	Ghost obstacles or erased cars	Use 2–5 past sweeps as motion cue; keep static layer confidence decay slow
Dashboard rendering slows the pipeline	Numbers look worse than reality	Visualiser runs in a separate process, subscribes to published maps, never blocks

Assumptions

- Ego-pose is available per sweep (from the dataset or an odometry node).
- Ground is locally planar within one 50 cm cell.
- A desktop GPU is available for development and the demo.

Open questions

- Is the 5 cm / 50 cm two-tier profile mandatory, or may we add intermediate tiers (for example 10 cm to 25 m) if they reduce the boundary jump?
- Is a live sensor demo expected, or is replay of a public dataset enough?
- Which hardware will the final evaluation run on?